Training queue / Design note

SOUP BACKEND ENGINEER | DESIGN RECORDED BEFORE IMPLEMENTATION

Boundary and guarantee

PostgreSQL is the durable queue and source of truth. An async FastAPI API, independent async worker processes, and SQLAlchemy repositories share the same transactional protocol. Redis Pub/Sub only reduces idle polling latency; every worker polls PostgreSQL periodically even if notifications are lost. SQLite's single-writer concurrency is a poor fit for the required eight-process contention test. PostgreSQL provides row locks, `SKIP LOCKED`, unique constraints, and transactional side effects.

Execution attempts are at least once; the committed database effect is exactly once for a succeeded job, and zero for a failed or cancelled job. The effect is an actual `job_effects` row, uniquely keyed by job ID, inserted in the same transaction as success. Computation before that transaction may run repeatedly. This guarantee does not extend to arbitrary files, emails, object storage, or external training systems: those require a destination-side idempotency/fencing protocol or a transactional outbox plus an idempotent consumer. Eventual completion assumes PostgreSQL recovers and a worker gets enough uninterrupted time; repeated crashes eventually exhaust a bounded attempt budget and produce `failed`.

Claim, recovery, and fencing

A short `READ COMMITTED` transaction selects one queued job or expired running job with `FOR UPDATE SKIP LOCKED`. It increments a monotonically increasing fencing token, records the attempt, and sets an owner and lease deadline using **PostgreSQL time**. No database transaction stays open during simulated training. Workers renew their lease periodically; a failed renewal stops the local computation. Any worker may reclaim an expired lease, so recovery requires no separate scheduler. A reclaim closes the previous attempt as expired. Expired jobs at the attempt limit become failed instead of remaining running forever.

Completion and renewal lock the job and then recheck state, owner, fence, and expiry against a fresh database timestamp. A paused worker returning with an old fence cannot mutate the new attempt or insert an effect. Completion inserts the effect, closes the attempt, and marks success in one transaction. A crash before commit rolls everything back; after commit the terminal job cannot be claimed again. The unique effect constraint is a final guard, not a way to silently swallow duplicate execution.

Cancellation and retries

Cancellation takes the same row lock as completion. If cancellation wins, it immediately marks the job cancelled, closes the active attempt, and clears its lease; heartbeats and completion are then rejected. If success/failure wins, cancellation returns HTTP 409 with the existing terminal state. Cancelling an already cancelled job is idempotent. Cancellation revokes permission to publish an effect immediately; stopping computation is cooperative and may lag until heartbeat/completion. No claim is made that a remote GPU process stops instantly.

Creating a job uses a database unique constraint on the required `Idempotency-Key`. A canonical, validated payload hash distinguishes a genuine replay (same job ID, HTTP 200) from conflicting reuse (HTTP 409). First creation returns 201 only after commit. A lost HTTP response is safe to retry. Terminal jobs retain their keys indefinitely in this exercise; any future retention policy must define the retry window. Pagination uses a `(created_at, id)` keyset cursor with a first-page high-water mark, not offsets; it is not a repeatable-read snapshot across requests.

Evidence and release gate

Use real PostgreSQL integration tests for simultaneous claims, lease expiry/zombies, heartbeat races, cancel/complete races, concurrent key reuse, rollback, server-clock authority, and isolation level. A subprocess harness must submit 10,000 unique jobs through HTTP, run eight real workers, kill a random live worker with SIGKILL roughly every two seconds and restart it, while duplicate POSTs and cancellations continue. Preserve raw logs from **every** invocation, including failures; compare the submitted manifest, API responses, database states, effect cardinality/content, and attempt history after draining. Run an additional offered-load HTTP experiment at 100 requests/second. These finite local tests do not establish production availability, sustained capacity, failover durability, or external side-effect safety. The verdict must reflect that limit.

Primary references

PostgreSQL locking clauses: `SKIP LOCKED` for queue-like consumers.

PostgreSQL READ COMMITTED: statement snapshots and concurrent updates.

SQLAlchemy async sessions: one session per concurrent task.

Redis Pub/Sub delivery semantics: notifications may be lost.